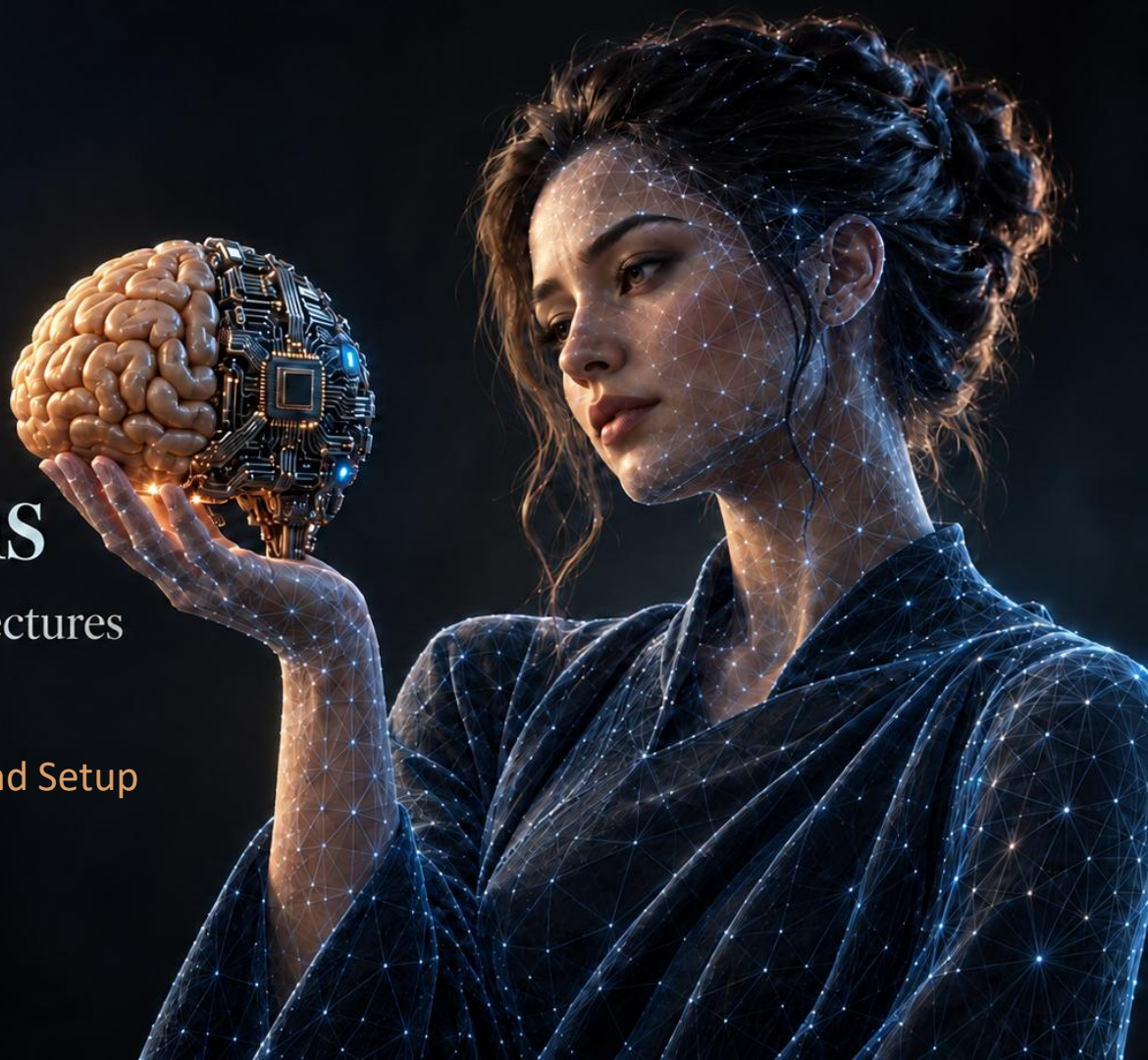
AI-powered Virtual Humans

LLM-Powered Cognitive Architectures

Module 0 — Course Introduction and Setup



Course Introduction



Course Materials

Everything you need is on GitHub.

Slides in PowerPoint and PDF
Notebooks and setup instructions
Browser avatar and Python agent



github.com/alessandropec/
virtual-humans-cognitive-architectures

Course Programme

Module 0 — Course Introduction and Setup

Module 1 — Cognitive Architectures and Large Language Models

1. Psychological Foundations of Cognitive Architectures
2. LLMs, Working Memory and Prompting
3. Introduction to LangChain

Module 2 — From Large Language Models to Virtual Humans

1. Agentic Frameworks and LangGraph
2. Memory and the Virtual Human
3. Tools and Coding Agents

Webinar — Further Practice

Virtual Humans Foundations

AI-Powered Virtual Humans

An optional course to take before this one.

Foundations of virtual humans.
Build an embodiment in Unity with speech, lip-sync, gestures and emotions.

Recommended background; not required for this course.

Course repository

Setting Up



First, We Need the Models

Providers host models and make them available through an API.

Google AI Studio	aistudio.google.com
Groq	console.groq.com
OpenRouter	openrouter.ai
Ollama Cloud	ollama.com

Create an account, generate an API key and save it in the YAML file.

Free access has limits: selected models, request quotas and account eligibility.

Cloud and Local Models

Cloud

Models run on a provider’s servers.
Access them through an API.
Use an account and an API key.

Local

Models run on your computer.
Ollama is one way to run them;
other tools are available.

In this course, we use cloud models with free tiers.

The Technology Stack

COMPONENT	PURPOSE	RESOURCE
VS Code	IDE and notebook editor	Open
Python	Runtime for all course code	Open
VS Code extensions	Python and Jupyter support	Open
Provider SDKs	Python clients for model providers	Open
LangChain	Model interfaces and prompt workflows	Open
LangGraph	State and agent orchestration	Open
Pydantic	Structured data and response validation	Open
FastAPI	API between the avatar and the agent	Open
Ollama	Local model server	Open

1. Download the Course Project

Open the course repository on GitHub.

Choose Code > Download ZIP, then extract the archive.

`virtual_human_course/`

Slides, notebooks and course requirements.

`virtual_human_project/`

Avatar, cognitive server and installers.

Keep both folders together in the extracted repository.

2. Install VS Code and Extensions

Install Visual Studio Code, then open Extensions.

Use the Extensions icon or Ctrl+Shift+X.

Install both Microsoft extensions:

Python

Run Python code and choose interpreters.

Jupyter

Open and run .ipynb notebooks.

Download VS Code

3. Install the Virtual Human

Follow the Installation guide for your computer.

Open the installer inside virtual_human_project.

Windows

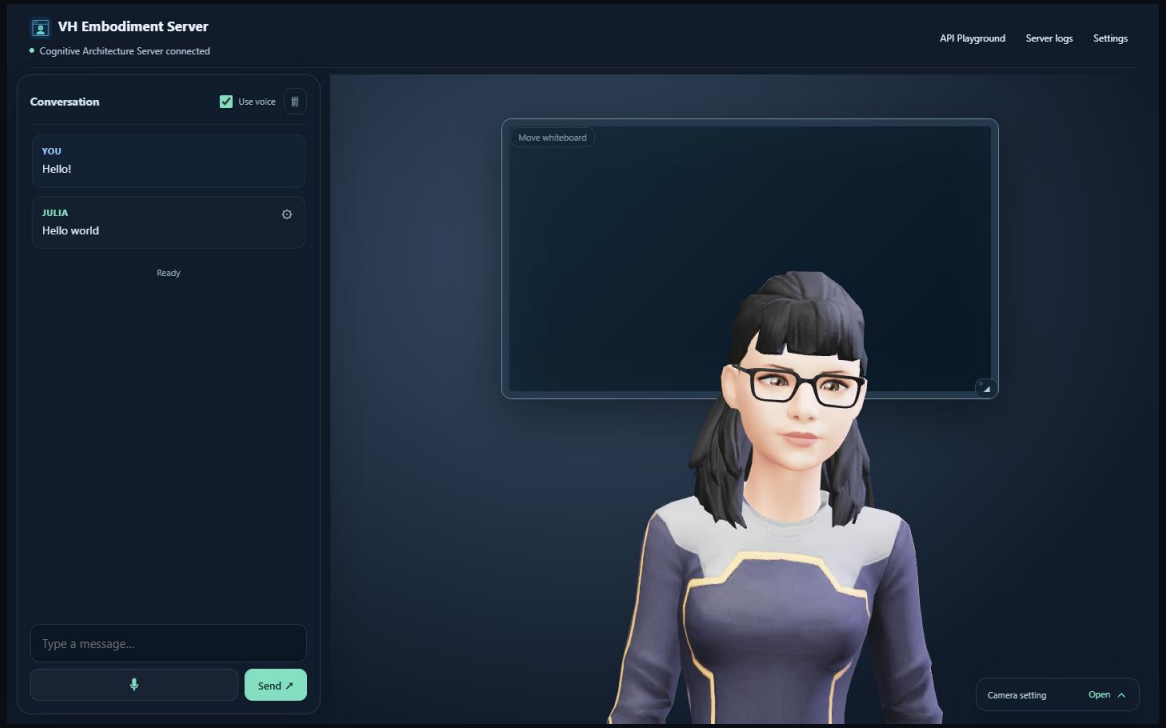
VirtualHuman-Setup.exe
Enable NVIDIA options only if setup detects a compatible GPU.

Mac

Install Virtual Human.app
CPU only; no GPU acceleration.

Click Install. Setup prepares Python, dependencies and speech models.

4. Launch and Test the Avatar



Check the result

Avatar visible

Connection green

Reply: Hello world

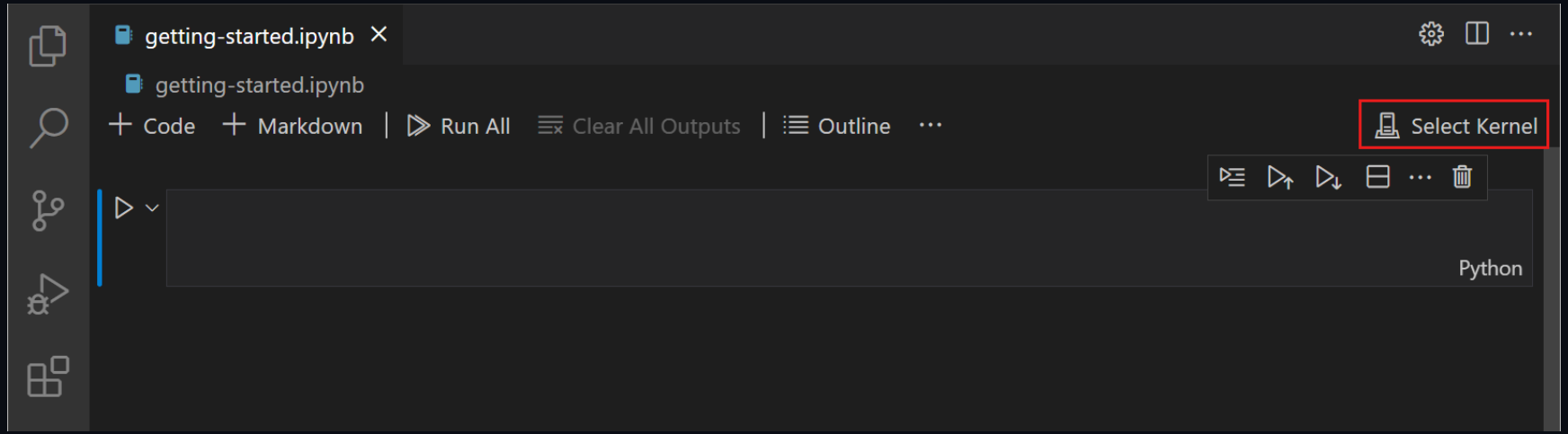
No API key needed.

[Agent guide]

5. Open the Notebook and Select the Kernel

In virtual_human_course, open:
Module_0/Chapter_01/Code/00_first_call.ipynb

Select Kernel: virtual_human_project/.venv-vh-course



6. Add Your Provider Key

Create an Ollama API key and use it in both folders.

For the notebook

`virtual_human_course/`

For the avatar's agent

`virtual_human_project/vh_cognitive_architecture_server/`

In both folders, copy llm_provider_api_key.example.yaml
to llm_provider_api_key.yaml. Paste your key and keep it private.

Try Ollama in the Notebook

Open the notebook and try a call with Ollama.

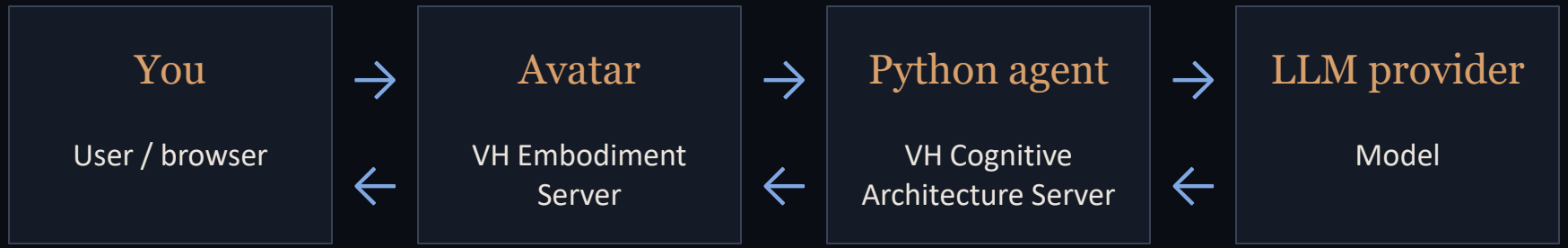
Run the key-loading cell, then the Ollama Cloud cells.

```
response = client.chat(
    model="gpt-oss:120b",
    messages=[{"role": "user", "content": "Say hello!"}],
)
print(response.message.content)
```

Open 00_first_call.ipynb

Try the Avatar

Your message goes to the agent; its answer returns to the avatar.



The mock always says “Hello world”.
With the naive agent, the LLM writes the answer.

Where to Change the Agent

Edit or create files in agents/ to define the agent's behaviour.

```
virtual_human_project/
  vh_cognitive_architecture_server/
    vh_cognitive_server.py
    agents/
      Module_0/Chapter_01/
        mock_agent.py
        naive_agent.py
```

Change the import to select the agent

Fixed reply

LLM reply

Save your files, then restart the server in Settings.

Does the avatar remember your name? Try it.

Why Do We Need a Cognitive Architecture?

YOU

Hi, I’m Alessandro. And you?

Why Do We Need a Cognitive Architecture?

YOU

Hi, I’m Alessandro. And you?

VIRTUAL HUMAN

Hi Alessandro, I’m PippoBot.
Nice to meet you!

Why Do We Need a Cognitive Architecture?

YOU

Hi, I’m Alessandro. And you?

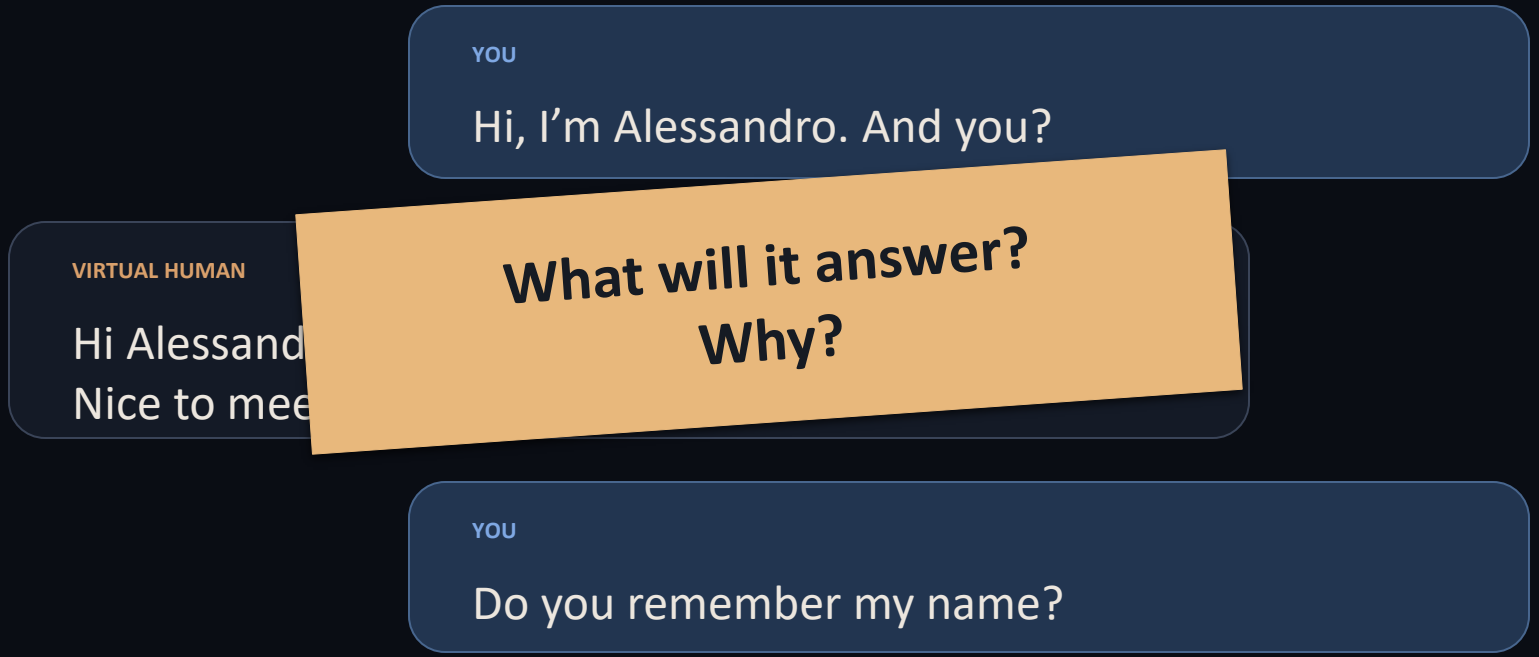
VIRTUAL HUMAN

Hi Alessandro, I’m PippoBot.
Nice to meet you!

YOU

Do you remember my name?

Why Do We Need a Cognitive Architecture?



Without Memory, Each Call Starts Again

YOU

Hi, I'm Alessandro. And you?

VIRTUAL HUMAN

Hi Alessandro, I'm PippoBot.
Nice to meet you!

YOU

Do you remember my name?

VIRTUAL HUMAN

I do not know your name. Could you tell me?

LLM calls are stateless: we must supply earlier information again.

From Stateless Calls to Cognitive Architecture

The model receives the question, not the conversation.

To remember the name

Memory model

Keep useful information
across interactions.

To build a virtual human

Cognitive architecture

Connect perception, reasoning,
memory and action.